

卢晓锋

个人信息

岗位：高级前端开发

经验：8 年

手机：15279189307

邮箱：luxiaofeng.code@gmail.com

学校：华东交通大学

专业：通信工程（本科）

工作经历

有赞科技有限公司	2024.06~至今	全栈开发
深圳市兔展智能科技有限公司	2020.07~2024.05	高级前端开发
深圳市秦丝科技有限公司	2019.05~2020.06	中级前端开发
深圳市猜猜城科技有限公司	2018.08~2019.05	初级前端开发

技能

- 前端**：HTML5、CSS3、JavaScript (ES6+)；**Vue**：Vue 3 升级与生态迁移、TypeScript、常用生态；**React**：业务开发与工程化实践；Webpack、Vite 构建与调试；包管理与工程化（Yarn→pnpm 等）
- 全栈**：Java、MySQL；业务接口与 SQL、前后端联调与问题排查
- 跨端**：小程序；Taro、Uni-app 等跨端框架
- 服务端与运维**：Node.js、Redis、Nginx、Docker 等
- Vibe Coding**：日常应用 Claude Code、Codex、Cursor 等 AI 编程工具，贯穿需求拆解、方案设计、编码实现、代码审查与问题排查，提升研发交付效率与代码质量
- AI 研发提效**：自研 MCP（辅助研发工作流，提升效率）；完善通用 Skill（日志 / DB / 接口 / 发布等）；OpenSpec 规范协作

项目经历

1、AI 研发工作流平台（dev-flow MCP Agent） 有赞科技 · 2025.03 - 2026.03

项目背景：线上问题排查、需求协作和发布验证需要在**日志、数据库、接口、发布**等多个内部系统间频繁切换，操作均在浏览器中进行，上下文割裂、路径长；AI 助手虽能辅助编码，但缺少直接访问内部研发系统的工具能力，难以串联完整研发闭环。

项目描述：主导设计并开发团队内 **dev-flow MCP Server**，将项目发布、天网日志查询、数据库查询、内部接口调用等原本需在浏览器手动操作的能力，封装为 AI 助手可编排调用的 **MCP Tool**；通过 **Cursor + Skill + MCP** 将“问题定位、上下文收集、方案拆解、代码修改、发布验证”串成 IDE 内研发工作流；**复用本机浏览器已登录会话，在工具侧延续同一登录态**，对接内部系统时自动完成鉴权。

业务价值：将排查与发布链路从「手动开多个 Tab → 浏览器操作 → 来回切换」压缩为在 **IDE 内由 AI 助手按步骤完成**；日志、数据、接口结果、需求文档与代码改动可在同一上下文中流转，缩短问题定位、修复验证和发布闭环，提升研发协作效率。

项目亮点：

- **AI workflow闭环**：围绕排障、需求协作、发布验证沉淀可复用 Skill，让 AI 助手不只生成代码，也能调用内部工具收集上下文并推进任务。
- **研发能力 MCP 化**：将日志、数据库、接口、发布等分散能力收敛为统一 MCP Tool，减少系统切换和人工复制粘贴。
- **无感登录提升可用性**：与浏览器共用同一登录会话，**接入即可调内部能力**，降低团队使用门槛。
- **Tool 粒度清晰可编排**：按单一职责拆分能力模块，便于 AI 助手基于任务步骤组合调用与扩展。

项目职责：主导 **MCP Server** 架构设计与核心工具开发，负责浏览器登录态复用、内部系统工具封装、Tool 输入输出协议设计、Cursor 接入联调，以及团队使用规范、OpenSpec 协作流程与 Skill 文档沉淀。

技术要点：

- **MCP Server**：基于 MCP 协议设计工具集，Tool 粒度对齐单一职责，供 AI 助手按步骤编排调用
- **无感登录**：工具侧延续浏览器会话；失效时引导完成企业统一登录（SSO）后恢复任务
- **能力模块**：发布（触发构建/部署流程）、天网日志（按 TraceId/时间/服务名查询）、DB 查询（只读，防误操作）、内部接口调用
- **Skill 编排**：将日志排查、DB 查询、接口调用、发布验证等高频路径沉淀为可复用 Skill，降低使用者对内部系统路径的记忆成本
- **协作串联**：与 **OpenSpec** 结合，在 IDE 内串联需求拆解、问题排查、代码修改与发布操作，项目文档同步飞书。

2、用户权益自动过期与标签增量刷新链路

有赞科技 · 2025.10 - 2025.11

项目背景：业务需要基于用户权益使用次数生成标签，用于用户分层与运营触达；但原链路缺少用户权益自动过期机制，随着权益数据持续累积，过期权益会影响用户当前标签命中结果，后端需要补齐自动过期能力，并提供稳定的标签增量刷新链路。

项目难点：权益自动过期后需要重新计算受影响用户的累计次数与标签结果；若全量扫描历史权益，DB 压力和任务耗时都会不可控。核心在于控制数据范围、批次内聚合去重，并保持权益状态与标签结果一致。

项目描述：负责用户权益自动过期与次数打标后端链路设计及 Java 落地，通过**定时任务 + 过期时间窗口筛选 + 每批 5000 条处理 + 用户维度聚合**，自动识别并处理到期权益，增量重算受影响用户标签；按批次模型推进，约 300 批可覆盖单日 150w 级过期记录处理。

业务价值：补齐用户权益自动过期能力，保障权益状态与标签命中结果及时一致；以增量任务替代全量重算，降低 DB 扫描压力，使过期标签刷新链路稳定可控。

项目亮点：

- **自动过期**：基于权益到期时间定时识别过期记录，完成权益状态刷新，并作为后续标签重算的数据入口。
- **增量重算**：以**昨日过期权益**圈定受影响用户，避免回扫全量历史数据。
- **分批处理**：按过期时间窗口分批拉取，每批处理 5000 条记录，按 300 批可覆盖约 150w 记录，降低单次查询、内存聚合和消息发送压力。
- **聚合去重**：按 **userId** 聚合累计次数，同一用户批次内只触发一次标签刷新，避免重复消息。

项目职责：负责后端方案设计与 Java 实现，完成权益自动过期、定时任务、MySQL 查询范围控制、分批处理、聚合去重、打标消息触发及联调上线。

技术要点：

- **过期识别**: MySQL 按权益到期时间查询待过期记录, 驱动权益状态刷新与后续标签重算
- **窗口筛选**: 按过期时间窗口圈定昨日过期记录, 缩小任务扫描范围
- **分批处理**: 按过期时间窗口分批处理, 每批控制在 5000 条, 避免单次加载过多数据
- **聚合去重**: 批次内按 `userId` 汇总次数, 命中阈值或区间后发送打标消息

4、前端依赖治理: Yarn 迁移 pnpm 与 Node 升级

有赞科技 · 2024.06 - 2024.11

项目背景: 仓库长期使用 **Yarn**, 随着依赖规模增长与 **Node** 版本演进, 本地安装、CI 构建及多人协作环境中逐步出现依赖解析不一致、幽灵依赖、旧包兼容性差等问题, 影响业务迭代与发版稳定性。

项目描述: 负责推进仓库从 **Yarn** 迁移至 **pnpm**, 并同步完成 **Node** 升级; 围绕依赖解析、锁文件收敛、安装脚本、CI 流水线及本地开发环境进行治理和适配, 结合 **pnpmfile** 编写兼容逻辑, 处理旧依赖在新包管理机制下的解析、提升与兼容问题, 保障迁移过程平滑落地。

业务价值: 统一本地与 CI 的依赖安装和构建环境, 降低因依赖解析差异、Node 版本不一致导致的构建失败与发版阻塞, 提升工程稳定性与团队协作效率。

项目亮点:

- **依赖治理前置**: 在迁移过程中梳理隐式依赖、版本冲突与旧包兼容问题, 推动依赖关系从“可安装”走向“可解释、可维护”。
- **包管理与运行时同步升级**: 将 **Yarn** → **pnpm** 与 **Node** 升级协同推进, 避免只升级单侧造成新的环境割裂。
- **兼容性优先的迁移策略**: 通过 **pnpmfile** 对旧依赖做解析与兼容适配, 降低存量包对迁移的阻力。
- **环境一致性治理**: 同步调整本地开发、安装脚本与 CI 流水线, 减少“本地可用、CI 失败”的情况。

项目职责: 负责 **Yarn** → **pnpm** 迁移方案落地与 **Node** 升级实施, 完成依赖问题排查、锁文件切换、安装与构建脚本适配、CI 配置调整、旧依赖兼容处理, 并协助团队完成本地与流水线环境对齐及问题排查。

技术要点:

- **包管理迁移**: 完成 **Yarn** → **pnpm** 切换, 调整锁文件、安装脚本与 CI 流程
- **依赖解析治理**: 处理幽灵依赖、版本冲突、peerDependencies 缺失等问题, 提升依赖结构稳定性
- **Node 升级**: 统一本地开发与流水线 Node 版本, 减少环境差异
- **兼容处理**: 通过 **pnpmfile** (如 `readPackage`) 对旧依赖进行兼容与解析适配, 降低迁移风险

5、标品 CRM 项目定制化

有赞科技 · 2024.12 - 2025.02

项目背景: 标品 **CRM** 需要服务不同行业客户, 不同行业对功能入口、页面交互和数据指标展示存在差异; 如果直接修改标品代码, 容易造成通用逻辑与定制逻辑耦合, 影响后续版本迭代和多客户交付。

项目描述: 参与标品 **CRM** 前端定制化方案建设, 基于 **Webpack** 模块解析链路设计全局定制文件替换机制: 在保留标品通用实现的基础上, 通过约定式定制后缀承接行业或客户差异; 构建时根据定制标识优先解析定制实现, 未命中则回退标品实现, 并配套调整 **TypeScript** 类型解析配置, 使类型检查、IDE 跳转与 **Webpack** 打包结果保持一致。

业务价值: 将标品能力和定制能力在工程层隔离, 避免多行业需求直接污染通用代码; 同一套工程可支撑不同客户包的构建与交付, 降低定制开发、回归测试和后续维护成本。

项目职责：参与前端定制化方案设计与落地，负责定制文件命名约定、Webpack 构建解析适配、TypeScript 类型解析配置、业务页面差异化开发、数据指标展示适配和联调验证，保障标品逻辑与定制逻辑边界清晰。

技术要点：

- **文件命名约定：**通过统一的定制后缀规范区分标品实现与行业差异，降低多人协作下的理解和维护成本
- **Webpack 定制打包：**构建阶段根据定制标识优先解析同路径定制文件，未命中时自动回退标品文件，输出面向不同客户或行业的定制包
- **TypeScript 类型识别：**同步调整 TS 类型解析规则，让类型检查与 IDE 跳转优先匹配当前定制环境，避免开发态与构建态结果不一致
- **定制影响面分析：**针对类型定义、组件入参等差异会沿调用链向上游传导的问题，结合依赖关系梳理定制影响范围，识别需要成套定制的面、组件和类型文件
- **定制逻辑隔离：**将行业功能、页面交互和数据指标差异收敛在定制文件及对应模块中，减少对标品主流程的影响

6、AI 数字人拜年小程序 兔展科技 · 2024.01 - 2024.02

项目背景：2024 年春节前约 15 天临时插入需求，时间紧、任务重，需在极短周期内在小程序内落地 AI 数字人拜年互动，支撑春节营销与品牌传播。

项目描述：担任前端负责人，推进 AI 数字人拜年小程序落地：根据组员能力拆解前端任务并安排分工；在需求频繁变化的情况下优先完成主框架搭建，再迭代页面与交互并推进数字人联调，保障春节前发版与活动期线上稳定性。

业务价值：在春节传播窗口内提供可分享、可互动的数字人拜年体验，支撑活动侧触达与转化；在短周期、强变更约束下仍完成可交付、可迭代的落地。

项目职责：担任前端负责人，负责技术方案与任务拆解、按成员能力分配任务与排期、主框架与核心页面落地、跨端联调与问题闭环，协调测试与发布。

技术要点：

- **主框架优先：**需求变化下先搭建可扩展的主框架，再并行迭代业务模块与联调
- 小程序活动页、分享链路及关键交互落地
- AI 数字人能力接入、联调与体验优化
- 临期场景下的版本管理与发布节奏、线上问题跟进

项目成果：

- 时间紧任务重下担任前端负责人，通过主框架先行与任务分工保障春节前按期上线
- 活动期保障前端侧稳定性与发版节奏

7、智慧零售 SaaS 平台项目 兔展科技 · 2022.06 - 2023.07

项目角色：前端负责人

项目背景：智慧零售 SaaS 平台长期运行在 Vue 1 + Webpack 1.x 老技术栈上，覆盖门店经营、营销活动、页面装修、小程序搭建等业务场景；其中核心装修模块耦合重、单文件体积大，构建与联调等待时间较长，已影响高频业务需求的交付效率。

项目描述：主导平台前端架构升级，完成 **Vue 1 → Vue 3**、**Webpack 1.x → Webpack 5** 迁移，并接入 **TypeScript**、**swc**、**thread-loader**、构建分析等工程化能力；同步重构核心「装修」模块，通过组件拆分、状态收敛、路由适配与迁移脚本降低存量系统升级成本。

项目职责：

- **主导平台前端架构升级：**为解决旧项目技术债与可维护性差的问题，主导完成 **Vue 1 → Vue 3**、**Webpack 1.x → Webpack 5** 技术迁移，并接入 **TypeScript**、**swc**、**thread-loader** 等工程化能力，使整体开发效率约提升 **20%**。
- **治理高复杂度核心模块：**针对装修模块渲染逻辑、配置面板与素材管理强耦合的问题，重新划分组件边界并收敛状态流转，推动核心模块从大体量、强耦合实现演进为职责清晰、可组合、可扩展的模块化架构，显著降低高频装修需求的扩展和维护成本。
- **优化构建与本地开发体验：**基于 **Webpack 5** 升级构建链路并兼容 **Vite** 本地开发模式，通过构建分析、并行编译和编译器替换，使 **Vite** 本地构建速度提升约 **90%**，**Webpack** 生产构建速度提升约 **36%**。
- **推动升级经验复用落地：**将组件拆分、路由兼容、迁移脚本与构建优化方案沉淀为可复用路径，为后续同类老项目升级提供实施参考，减少重复摸索成本。

项目成果：

- **Vue 3** 生态落地后，整体开发效率约提升 **20%**，开发、联调与提测等待时间明显缩短。
- 核心装修模块完成组件化与工程化重构，装修类需求扩展和改版成本明显下降。
- 存量老项目迁移路径可复用，为后续同类模块升级提供了迁移脚本、路由兼容与构建优化经验。

8、前端工程化基线与研发效能体系建设

兔展科技 · 2021.05 - 2022.10

项目背景：公司多条业务线在新项目初始化时，监控、Lint、CI、埋点等基础能力主要依赖人工按文档接入，配置口径不统一、重复投入成本高；同时已有埋点 SDK 初始化参数复杂，且 **Web / 小程序** 等多端接入方式存在差异，影响新业务项目的启动效率。

项目描述：主导建设公司级前端工程化基线体系，通过 **CLI + 远程模板 + 能力预设 + 版本管理** 将项目初始化、质量规范、监控接入、CI 配置和埋点能力标准化、自动化；在不改变原有埋点 SDK 能力边界的基础上，封装多端接入适配层，支持页面生命周期自动上报与 **data-track** 声明式点击埋点，降低业务代码侵入和重复接入成本。

项目职责：

- **设计工程基线落地方案：**将分散在文档中的工程规范沉淀为可执行 **CLI** 流程，统一项目创建、基础配置、规范接入和示例代码生成链路。
- **建设模板化初始化体系：**将远程模板与 CLI 解耦并进行版本化管理，支持按项目类型拉取不同工程基线，满足业务后台、小程序等场景的差异化初始化需求。
- **封装多端埋点接入能力：**基于公司已有埋点 SDK 做二次封装，屏蔽复杂初始化参数和多端接入差异，提供页面生命周期自动上报与 **data-track** 元素绑定能力。
- **推动工程规范自动化执行：**在创建流程中预置监控、Lint、CI、埋点等基础能力，将“按文档逐项配置”转为脚手架内置能力，提升团队执行一致性。

技术要点：

- **CLI 子命令体系：**基于交互式命令组织项目创建、模板选择、能力注入和配置生成流程，降低后续新增工程能力的接入成本。
- **远程模板管理：**通过远程模板和版本管理维护不同项目类型的工程基线，使模板更新与 CLI 发布节奏解耦。

- **能力按需注入**：在初始化阶段按需生成监控、Lint、CI、埋点等配置与示例代码，减少业务侧手工复制和遗漏风险。
- **埋点低耦合接入**：通过统一初始化封装、页面生命周期自动上报和 **data-track** 声明式绑定，减少业务逻辑中的手写上报代码。

项目成果：

- 新项目从创建到本地可运行耗时由约 **2 小时** 压缩至 **10 分钟内**，启动效率提升约 **90%**。
- 工程规范由“文档约束 + 人工配置”升级为“脚手架自动注入 + 模板统一维护”，团队项目初始化口径更加一致。
- 埋点接入模板落地后，业务侧重复接入与手写上报成本下降，相关开发效率约提升 **20%**。
- 通用脚手架累计支撑数十个项目初始化，工程基线与埋点能力可在多条业务线复用和推广。

9、小程序 DevOps 发布平台建设

兔展科技 · 2020.12 - 2021.04

项目背景：小程序发布依赖人工操作开发者工具，版本记录与上传流程缺少统一管控，多人并行发版时最容易出现**版本互相覆盖**，进而引发错版、返工打包和验收链路反复确认。

项目描述：建设面向多小程序项目的工程化发布平台，通过 **CLI + 发布配置 + 取码平台** 将构建、上传、预览、发布前检查、版本记录和二维码协作标准化；同一套发布流程可在多个小程序工程复用，支持失败重试和发布记录追踪，重点降低多人协作下的版本覆盖风险。

业务价值：将小程序发布从「人工操作开发者工具」升级为「按配置执行的标准化流程」，提升发版可控性与可追溯性；业务与测试可通过取码平台自助获取预览码、体验码，减少研发反复打包和即时通讯传码成本。

项目职责：负责整体方案与核心开发，完成 **CLI** 发布链路、工程配置解析、构建/上传/预览/检查流程编排、失败重试与日志记录；搭建取码平台并打通发布产物、版本记录和二维码分发链路。

技术要点：

- **可配置发布流水线**：以配置文件描述工程路径、多目标构建、上传、预览和发布前检查，步骤可脚本化、可复用、可重复执行
- **发布可追溯**：集中管理版本号、发布记录和执行日志，降低多人并行开发下的版本覆盖、错版和口头同步成本
- **取码协作平台**：将发布产物与二维码能力收口到统一入口，服务研发、测试、业务验收和企业微信分发场景

项目成果：

- 发布流程平台化后，**研发侧**准备与执行发版相关工作的效率约提升 15%
- 版本与发布信息集中可追溯，降低版本互相覆盖等核心协作风险，**协作效率**约提升 60%
- 取码平台减少反复打包与传码成本，**联调与业务验收**相关效率约提升 30%